Eliminating Timing Channels in Microkernels

Integrating Time Protection Mechanisms into FreeRTOS

Simon V. Brodersen, 24-06-2026

Timing Channels: The Problem

- Timing channels leak information through observable execution time
- Real-world attacks:
  - Cryptographic key extraction; e.g., OpenSSL (CVE-2025-9231)
  - Cross-VM information leakage in cloud computing
- Difficult to eliminate:
  - Hard to diagnose
  - Inherent tension between performance (sharing) and security (isolation)
- **Core challenge:** Closing timing channels at the OS level

Thomas Ristenpart et al. "Hey, You, Get off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds". In: *Proceedings of the 16th ACM Conference on Computer and Communications Security*. CCS '09: 16th ACM Conference on Computer and Communications Security 2009. Chicago Illinois USA: ACM, Nov. 9, 2009, pp. 199–212. ISBN: 978-1-60558-894-0. DOI: 10.1145/1653662.1653687. URL: https://dl.acm.org/doi/10.1145/1653662.1653687 (visited on 02/04/2026)

Research Question & Contributions

Research Question:

Can time protection mechanisms be integrated into a mainstream OS to mitigate timing channels?

Contributions:

1. A **domain-level scheduler** partitioning execution into fixed time slices
2. **Temporal isolation** via the `fence.t` instruction on domain switches
3. **Evaluation** on QEMU showing constant-time domain scheduling

Background: Timing Channels

Three categories of timing channels in operating systems:

- **Scheduler-based:** Priority decisions leak runnability info
- **Shared-state:** Caches, TLBs, branch predictors retain cross-task state
- **Delay-based:** Interrupts and locks delay execution predictably

Isolation as the countermeasure:

- *Spatial:* Memory partitioning (MMU / MPU)
- *Temporal:* Ensuring that shared resource usage in one domain does not affect another

The `fence.t` Instruction

- Hardware-assisted temporal isolation for RISC-V
- Flushes on-core microarchitectural state on domain switches
- Deterministic execution time — independent of hardware state
- Overhead in seL4: only **0.7%**
- Software-supported variant (`fence.t.s`) for out-of-order cores: **1.0%**

Nils Wistoff et al. "Systematic Prevention of On-Core Timing Channels by Full Temporal Partitioning". In: *IEEE Transactions on Computers* 72.5 (May 2023), pp. 1420–1430. ISSN: 2326-3814. DOI: 10.1109/tc.2022.3212636. URL: http://dx.doi.org/10.1109/TC.2022.3212636, Nils Wistoff, Gernot Heiser, and Luca Benini. "Fence.t.s: Closing Timing Channels in High-Performance Out-of-Order Cores Through ISA-Supported Temporal Partitioning". In: *Applications in Electronics Pervading Industry, Environment and Society*. Ed. by Massimo Ruo Roch et al. Cham: Springer Nature Switzerland, 2025, pp. 269–276. ISBN: 978-3-031-84100-2. DOI: 10.1007/978-3-031-84100-2_32

User-level mitigations

- Constant time application code
- **An example:** FaCT is a domain-specific language focused on cryptographic code
- The programmer marks secret information; FaCT creates constant-time code through information flow analysis
- Safe code at the C level

Shortcomings:

- Variable execution time of operators; for example, division time depends on the numerator
- Compiler introduced optimizations
- Is scheduling sensitive information?

The Missing OS Abstraction

seL4 Time Protection: Introduced the notion of time protection at the OS level.

- Built as a modification to the existing seL4 kernel
- Copies the kernel code into each security domain
- Flushes shared hardware that cannot be spatially partitioned
- Cache coloring to partition higher-level caches

Qian Ge et al. "Time Protection: The Missing OS Abstraction". In: *Proceedings of the Fourteenth EuroSys Conference 2019*. EuroSys '19. New York, NY, USA: Association for Computing Machinery, Mar. 25, 2019, pp. 1–17. ISBN: 978-1-4503-6281-8. DOI: 10.1145/3302424.3303976. URL: https://dl.acm.org/doi/10.1145/3302424.3303976 (visited on 02/03/2026)

S3K capability-based RTOS

S3K provides a dynamic, capability-based approach.

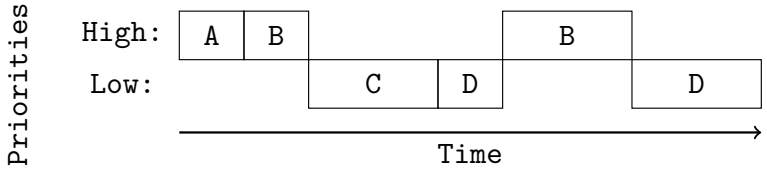
- A capability-based model that extends to time slices
- A time-driven, capability-based scheduler with systematic use of `fence.t`
- Predictable, constant-time system calls
- Scratchpad memory for kernel code execution (backed by L1 cache, flushed on scheduling)

The Gap: No prior work integrates these mechanisms into an *existing* mainstream OS like FreeRTOS.

Henrik Karlsson and Roberto Guanciale. "Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning". In: *2025 IEEE Real-Time Systems Symposium (RTSS)*. 2025 IEEE Real-Time Systems Symposium (RTSS). Dec. 2025, pp. 68–81. DOI: 10.1109/RTSS66672.2025.00015. URL: https://ieeexplore.ieee.org/document/11315078 (visited on 02/04/2026)

Vulnerability: Priority-Based Scheduling

FreeRTOS uses a priority-based preemptive scheduler, which is inherently unfair and vulnerable to timing attacks.



The channel: A high-priority task signals 0 or 1 by sleeping or running, and the low-priority task measures its own wake time.

Chien-Ying Chen et al. "A Novel Side-Channel in Real-Time Schedulers". In: *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). Apr. 2019, pp. 90–102. DOI: 10.1109/RTAS.2019.00016. URL: https://ieeexplore.ieee.org/document/8743170/ (visited on 02/09/2026)

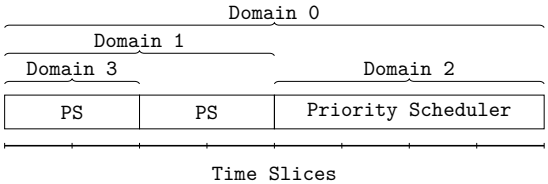
Possible Solutions

We considered several approaches to address priority-based timing vulnerabilities.

- **Fixed execution time:** Each priority would have a known fixed execution time and scheduling. Significant idle time
- **Introduction of Domains:**
  - A domain is a group of tasks, which the scheduler treats as a single environment
  - Only partitioning across domain boundaries allows for intra-domain channels
  - Requires the notion of time slices, which is a fixed execution time serving as the smallest unit of "time" for the scheduler
  - Should domains be dynamic or static?

Design: Domain Scheduling

Chosen approach: Domain scheduling



- Initially, all time slices are given to Domain 0
- Domain 0 gives up fixed time slices to other domains on creation
- Tasks within a domain use standard priority scheduling
- Allows for a trade-off between the throughput (priority) and partitioning (domain)

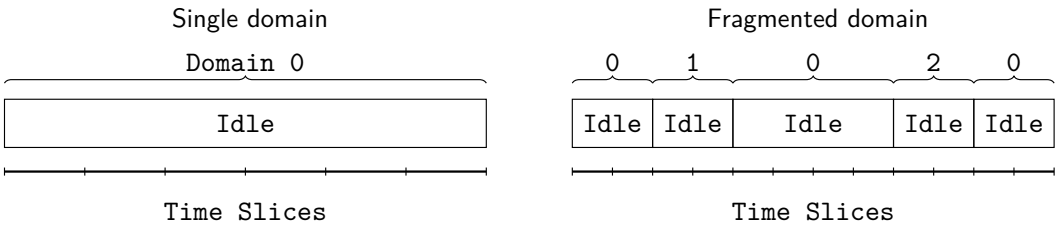
Implementation: Extending the FreeRTOS API

FreeRTOS MPU creates restricted tasks via `xTaskCreateRestricted`. We extend the task creation API with a domain parameter.

```
typedef struct xDOMAIN_PARAMETERS {
  uint32_t ulSliceIndex;   // Start time slice
  uint32_t ulSliceLength;  // Number of slices
} DomainParameters_t;
```

- When creating a restricted task, one must pass the start of the group of time slices and their length
- If no `DomainParameters_t` is passed, it creates the tasks within the current domain

Implementation: Constant-Time Domain Creation



- `xDomainInfo` array indexed by domain id, contains the start index of the domain in `xDomains`
- `xDomains` array indexed by time-slice start
- `uxNext` linked list finds next block of the current domain in **constant time**

Implementation: Domain Switching

FreeRTOS's basic time unit is a *Tick*, incremented on each machine timer interrupt in the RISC-V port.

Idea: Tie time slices to *Tick*

- Allow for *Tick* to be used by the priority-based scheduler
- Check for end of time slice when incrementing Tick
- Flush microarchitecture with `fence.t` on domain switch

Implementation: Managing Cross-Domain Queues

FreeRTOS `Queues`: primary tool for communication between different tasks.

Normal Queue Operation

- `Queue` has an attached **waiting** list
- Tasks block on the waiting list when requesting messages
- On message send, task moves from waiting $\rightarrow$ ready

Domain Complication

- Each domain has its own **ready** queue
- A blocked task must be routed to the correct domain's ready queue

Solution: Track each task's domain ID; on wake-up, enqueue it to the correct domain's ready queue.

Introduces timing channels for tasks with access to the `Queue` object.

Evaluation: QEMU Platform & Limitations

Why QEMU?

- Full system emulation of RISC-V virtual machine
- Allows approximating the time each domain runs before scheduling

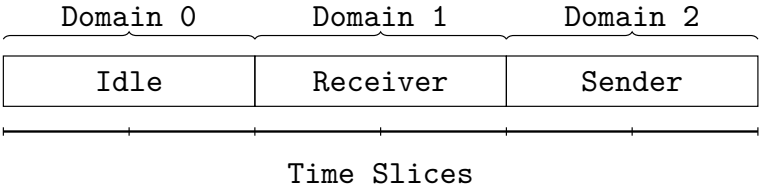
Shortcomings:

- No cycle-accurate emulation: microarchitectural state (caches, TLBs, predictors) is *not* modeled
- Timing channels from shared microarchitectural state cannot be diagnosed

Implication: Only constant-time execution paths can be verified. Real hardware with `fence.t` support (or gem5) is needed to fully evaluate temporal isolation.

Evaluation: Blinky Demo

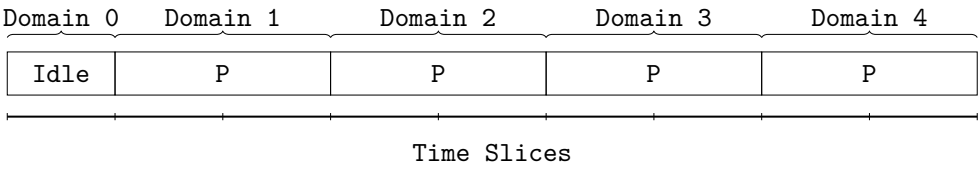
Two tasks in separate domains communicating via a Queue:



- Sender scheduled at domain tick 4, Receiver at domain tick 2
- Round-trip time measured with `rdcycle`: **50,000 cycles ± 1 cycle**
- Demonstrates constant-time domain switching

Evaluation: Multi-Domain Demo

4 domains, each with a privileged task that creates a higher-priority unprivileged task:



- Domain ticks are deterministic (1, 3, 5, 7, ...)
- Priority scheduler still works within each domain

Conclusion

What I showed:

- Domain-level scheduler achieves constant-time scheduling in QEMU
- Trade-offs
  - **Complexity:** A significant amount of complexity has been added to the code. Future work on FreeRTOS would be more cumbersome with the extra code complexity.
  - **Performance:** Given the implementation makes use of idle time, total performance is most likely reduced significantly. Further, more memory accesses are needed for the domain scheduler to function, so while asymptotically the same, it incurs a larger overhead.

Conclusion: Domain isolation is possible and functional in an existing OS.

Future work

- Evaluation on real hardware with `fence.t` support (or gem5)
- Review system calls: critical sections delay scheduling, only user-level protection
- Introduce a two-level locking system.
  - FreeRTOS has a notion of "critical section". These sections grab a global lock to ensure consistency of the priority-based scheduler.
  - Currently the domain-level scheduler uses the same lock.
  - **Idea:** Allow for a separate locking mechanism for cross-domain consistency. This would allow scheduling a new domain while holding the priority-based scheduler lock.
- Remaining channels: interrupts, higher-level caches

Toward a Safe General-Purpose OS

While a full domain-level scheduling setup does function, it performs worse and adds complexity. The primary goal of a general-purpose OS is throughput; the domain-level scheduler seems like overkill. An alternative is to introduce aspects.

- Allow for the creation of "secure" tasks, which would be partitioned. Two general aspects could be tested
  - Flush before and after a secure task, and allow for partitioned higher-level caches.
  - Introduce the Scratchpad Memory. Here one could either restrict the task to this memory footprint, or use the Scratchpad Memory as a buffer, which we can index with secrets. This would require loading all data into the SPM.